Engineering Progression: Logic, Scope, and Data Normalization

Summary
A WIP document to help train myself in the Python programming language, from the basics to more advanced coding. The AI provided the Practice Questions. I developed the code in the Answer, tested it in VS Code, and added it to this doc. This is to fill a skill gap in my resume, as I am often assumed to be an engineer based on my track record. I also used my technical writing skills to design and write the document.

Author: Melinda Cuffy
Status: WIP
Created: Apr 25, 2026
Updated: Apr 25, 2026

Intended Audience: Melinda Cuffy, Ben Treynor

Visibility: Need to Know

Technical Progression: From Policy to Code

Over the past several weeks, I have transitioned my background in global program governance into a functional engineering toolkit. Using Gemini as a senior-architect mentor to provide real-world infrastructure scenarios, I have rebuilt my logic foundations from the ground up in VS Code.

This document outlines twelve distinct phases of technical development. Each phase represents a specific mastery in "Clean Code" principles—from handling raw data streams to creating robust, case-insensitive security audits. My focus throughout has been on ensuring that every script functions as its own documentation, prioritizing code readability for high-stakes engineering environments.

Conditional Filtering and List Population

In this phase, I mastered the process of creating a filtered subset from a primary dataset. By initializing an empty list (adults) and using an if statement inside a loop, I learned how to evaluate each item against a specific threshold. The .append() method was then used to selectively populate the new list with only the data points that met the criteria. This demonstrates the fundamental logic required to isolate actionable records from a raw data stream.

Question 1: Filtering a List

This is how you take a big list of data and pull out only what you want.

Example

I have a list of prices. I only want the ones that are **under $50**.

```python
prices = [10, 65, 20, 80, 45]
cheap_items = []

for p in prices:
    if p < 50:
        cheap_items.append(p)

print(cheap_items)
```

Practice

You have a list of ages: [15, 22, 12, 40, 17, 31]. These represent individuals requesting entry at a site security checkpoint.

Write a script that:

1. Starts with ages = [15, 22, 12, 40, 17, 31].
2. Starts an empty list called adults.
3. Loops through ages and uses an if statement to find numbers 18 or older.
4. Appends those numbers to the adults list.
5. Prints adults.

Answer

Python

```python
# initializes variables for a list of ages, along with an empty variable for adults over the age of 18.
ages = [15, 22, 12, 40, 17, 31]
adults = []

# loops through the list of ages to determine if any of the numbers are greater than or equal to 18.
# adds any number greater than or equal to 18 to the adults list.
for a in ages:
    if a >= 18:
        adults.append(a)

# prints adult ages greater than or equal to 18.
print(adults)
```

Data Transformation and List Building

> In this phase, I moved beyond just filtering data. I learned how to "transform" values by creating a secondary list of (final_scores). Inside the loop, I defined a calculation (s + 5) and used the .append() method to capture the results of that calculation. This allowed me to preserve the original raw scores while generating a completely new set of processed data.

Question 2: Modifying Data (The Loop "Map")

Sometimes in ML, you need to change your data before you use it (like converting Celsius to Fahrenheit). This is called "mapping."

Example

I have a list of prices in dollars, and I want to add a **$2 tax** to every single one of them.

```python
prices = [10, 20, 30]
taxed_prices = []

for p in prices:
    new_price = p + 2
    taxed_prices.append(new_price)

print(taxed_prices)
```

Practice

You have a list of scores: [80, 90, 70]. The teacher is feeling generous and wants to give everyone a 5-point bonus.

Write a script that:

1. Starts with scores = [80, 90, 70].
2. Starts an empty list called final_scores.
3. Loops through scores and adds 5 to each score.
4. Appends the new result to the final_scores list.
5. Prints final_scores.

Answer

```python
# lists the variables for current and final test scores.
scores = [80, 90, 70]
final_scores = []

# defines a variable for 5 bonus points to be added to each current test score.
# adds final numeric test scores to final_scores variable.
for s in scores:
    bonus = s + 5
    final_scores.append(bonus)

# prints a list of final test scores.
print(final_scores)
```

The Accumulator Pattern (The "Bucket" Logic)

> This phase was about mastering the Running Total. I initialized a variable (total_badges) at zero to act as my "Bucket." Inside the loop, I used the logic total_badges = total_badges + badge. This tells the computer to take whatever is currently in the bucket, add the new entrance data to it, and pour the whole sum back into the bucket. This is the foundation for calculating system-wide metrics from individual site reports.

Question 3: The "Totalizer" (Summing Data)

Now that you're comfortable moving and changing data, let's learn how to **aggregate** it. This is a huge part of ML (like calculating the total "Error" or "Loss").

The Logic: You start with a "bucket" (a variable) set to **0**, and every time you loop, you throw a value into that bucket.

Example

I want to find the total number of hours worked across three days.

```python
hours = [8, 7, 9]
total_hours = 0  # Start the bucket at zero

for h in hours:
    total_hours = total_hours + h

print(total_hours)
```

Practice

You are reviewing the **physical data security** at a Google site. You have a list of how many security badges were issued at three different entrances: [10, 15, 20].

Write a script that:

1. Starts with badges = [10, 15, 20].
2. Starts a "bucket" called total_badges at **0**.
3. Loops through badges and adds each count to total_badges.
4. Print total_badges.

Answer

```python
# lists the variable for badge counts from three site entrances.
# also lists a variable for the total sum of badges, starting at
zero.
badges = [10, 15, 20]
total_badges = 0
```

```
# loops through the badges from each entrance to calculate the
total sum of badges.
# drops the badges from each entrance into the bucket.
for badge in badges:
    total_badges = total_badges + badge

# prints the total sum of all the badges from all entrances.
print(total_badges)
```

Security Thresholds and Scope

> In this phase, I applied the accumulator logic to a security scenario, but with a focus on Scope. I learned that by placing the if/else statement outside and after the loop, the program waits for the full count to be completed before issuing a final status report. This ensures the system only triggers a "System Lockout" alert based on the aggregate total of all failed attempts, rather than reacting to each individual data point.
>
> This was a major breakthrough. I learned that where you place your code determines when it triggers. By moving the if statement outside the loop, the system provides a final verdict rather than a noisy per-item report.

Question 4: The Logic Gate

This pattern is essential for reducing "noise" in system monitoring. By aggregating data before triggering an alert, you ensure the system only responds to the total state rather than individual fluctuations.

Example

I want to see if the total number of items in a shipment exceeds my vehicle's capacity of 100.

```python
shipment = [30, 40, 50]
total_items = 0

for box in shipment:
    total_items = total_items + box
if total_items > 100:
    print("ALERT: Capacity Exceeded")
else:
    print("Status: Load Secure")
```

Practice

You are reviewing server logs for "Failed Login Attempts." If the total number of failures reaches a specific threshold, you must trigger a lockout.

Write a script that:

1. Starts with failed_attempts = [2, 5, 1, 8].
2. Starts a "bucket" called total_failures at 0.
3. Loops through failed_attempts and adds each count to total_failures.
4. After the loop finishes, uses an if/else statement:
5. If total_failures is 15 or greater, print "SECURITY ALERT: System Lockout Triggered".
6. Otherwise, print "Status: System Secure".

Answer

```python
# initializes the variables for failed login attempts at different times of day,
# additionally, defines and implements the starting point of zero for the total number of failures variable.
failed_attempts = [2, 5, 1, 8]
total_failures = 0
```

```
# starting at zero, loops through failed attempts and adds all
attempts to failed attempts.
for attempt in failed_attempts:
    total_failures = total_failures + attempt

# prints out alerts for whether the total number of failed login #
attempts is within or outside a secure range.
if total_failures >= 15:
    print("SECURITY ALERT: System Lockout Triggered")
else:
    print("Status: System Secure")
```

Error Counting and String Matching

> This phase involved isolating specific string values within a list to perform a targeted count. Instead of adding up raw numbers, I used a conditional if statement inside the loop to look for the specific string "Error." Every time the loop encountered a match, it incremented the error_count by one. This demonstrates the ability to filter through status reports and extract specific metric totals from a larger dataset.

Question 5: Dealing with "Strings" (Text Data)

In your role as an **AI Senior Reviewer**, you dealt with a lot of text (strings). Sometimes we need to filter data based on words to find errors or specific labels.

Example

I have a list of fruits. I only want to find and count the "Apples."

Python
```
inventory = ["Apple", "Banana", "Apple", "Orange"]
```

```
apple_count = 0

for fruit in inventory:
    # Use == to ask "is it equal to?"
    if fruit == "Apple":
        apple_count = apple_count + 1

print("Total Apples found:")
print(apple_count)
```

Practice

You are auditing a list of Google API status reports: ["Success", "Error", "Success", "Success", "Error"].

Write a script that:

1. Starts with api_reports = ["Success", "Error", "Success", "Success", "Error"].
2. Starts a bucket called error_count = 0.
3. Loops through the list using **Descriptive** naming (for report in api_reports:).
4. Inside the loop, uses an if statement to check if the report is **equal to** "Error".
5. If it is an "Error", adds **1** to the error_count.
6. Print the error_count at the very end.

Answer

Python

```
# initializes the variables for successes and errors found in
Google API status reports.
# implements zero as a starting point for the error count.
api_reports = ["Success", "Error", "Success", "Success", "Error"]
error_count = 0
```

```
# checks for and calculates the sum of all "Error" outputs.
for report in api_reports:
    if report == "Error":
        error_count = error_count + 1

# prints the total number of errors found in the status report.
print("Total Errors Found:")
print(error_count)
```

Indexing Nested Lists

> This phase focused on navigating "lists within lists" to extract specific data points. I learned how to use index numbers to reach into a record and assign specific values to variables, such as entry[1] for names and entry[2] for status. By deconstructing each nested list inside the loop, I was able to run a conditional check on individual attributes and generate personalized outputs based on the status of each specific record.

Question 6: Nested Data Extraction

In system administration, data often arrives in "rows" where each row contains multiple pieces of information (like a spreadsheet). To process this, you have to "index" into the specific column you need.

Example

I have a list of employees and their departments: [["Alice", "HR"], ["Bob", "IT"]]. I want to print only the names.

Python
```
employees = [["Alice", "HR"], ["Bob", "IT"]]
```

```
for person in employees:
    name = person[0]  # Index 0 is the first item (the name)
    print(name)
```

Practice

You are reviewing site logs from different Google data center locations. Each entry contains the [Location, Staff Name, Status].

Write a script that:

1. Starts with logs = [["Huntsville", "Sarah", "Complete"], ["Dublin", "Mike", "Pending"], ["Huntsville", "Joe", "Pending"]].
2. Loops through the logs list.
3. Assigns entry[1] to a variable called name.
4. Assigns entry[2] to a variable called status.
5. Uses an if/else statement:
6. If the status is "Pending", print name + " needs a follow-up."
7. Otherwise, print "Complete".

Answer

Python

```
# initializes variables for information found in logs.
logs = [["Huntsville", "Sarah", "Complete"], ["Dublin", "Mike",
"Pending"], ["Huntsville", "Joe", "Pending"]]

# defines separate variables for name and status.
# loops through each person's record to identify their status.
# prints an output for whether the person in the record needs # a
follow up, or whether they show as completed.
for entry in logs:
    name = entry[1]
```

```
status = entry[2]
if status == "Pending":
    print(name + " needs a follow-up.")
else:
     print("Complete")
```

Multi-Condition Governance Audit

> This phase demonstrated the use of logical operators to enforce specific site policies. By using the and operator within a loop, I learned how to filter data based on multiple criteria simultaneously—in this case, both the geographical location and the badge status. This allows for a more granular audit, ensuring that only high-priority records (Huntsville with a Missing status) are flagged for urgency while other compliant records are processed as "Site OK."

Question 7: Using Multi-Condition Logic

In governance, a single factor rarely triggers an alert. Usually, you are looking for a specific combination of factors (e.g., "The site is Huntsville" AND "The badge is missing"). Using the and operator allows you to build these specific "Logic Gates."

Example

I want to find people who are from the "Sales" department AND have "Manager" in their title.

Python
```
staff = [["Sales", "Manager"], ["IT", "Manager"], ["Sales",
"Associate"]]
```

```
for person in staff:
    dept = person[0]
    role = person[1]

    if dept == "Sales" and role == "Manager":
        print("Executive Found")
```

Practice

You are auditing onboarding data for specific site compliance. You need to flag any records specifically for the Huntsville site where a badge is still missing.

Write a script that:

1. Starts with onboarding_data = [["Huntsville", "Sarah", "Got Badge"], ["Dublin", "Mike", "Missing"], ["Huntsville", "Joe", "Missing"], ["Singapore", "Ling", "Missing"]].
2. Loops through the onboarding_data list.
3. Assigns entry[0] to site, entry[1] to name, and entry[2] to badge_status.
4. Uses an if/else statement with the and operator:
5. If the site is "Huntsville" AND the badge_status is "Missing", print "Urgent".
6. Otherwise, print "Site OK".

Answer

Python

```
# defines a list for records found in onboarding data.
onboarding_data = [["Huntsville", "Sarah", "Got Badge"], ["Dublin",
"Mike", "Missing"], ["Huntsville", "Joe", "Missing"], ["Singapore",
"Ling", "Missing"]]

# defines variables for information found in each record.
# loops through each person's record to check location and status.
```

```
# Flags a record based on if a badge is missing for a person based
in huntsville.
for entry in onboarding_data:
    site = entry[0]
    name = entry[1]
    badge_status = entry[2]
    if site == "Huntsville" and badge_status == "Missing":
        print("Urgent")
    else:
        print("Site OK")
```

Multi-Condition Reporting and F-String Formatting

In this phase, I transitioned from basic output to dynamic reporting. By implementing f-strings (formatted string literals), I learned how to embed variables directly into strings, which improves code readability and output professionality. This phase also reinforces multi-condition logic by auditing data against two different thresholds—geographical location and security clearance—to produce a tailored notification for specific infrastructure events.

Question 8: Dynamic Infrastructure Auditing

> This is how you generate specific, readable alerts instead of generic messages. By using f-strings, the system provides context (who, where, and what) in a single line of output without needing complex concatenation.

Example

I have a list of employees and their office locations. I want to print a custom welcome message for everyone in the "New York" office.

```python
# utilizing an f-string to inject variables into a sentence
name = "Alice"
office = "New York"

if office == "New York":
    print(f"Welcome to the team, {name}! Enjoy the {office}
office.")
```

Practice

You are auditing global security clearances. You need to flag any "Contractor" in the "London" office for a mandatory secondary screening.

Write a script that:

1. Starts with staff_audit = [["London", "James", "Contractor"], ["Paris", "Marie", "Employee"], ["London", "Kiran", "Employee"], ["London", "Elena", "Contractor"]].
2. Loops through the staff_audit list.
3. Assigns entry[0] to location, entry[1] to name, and entry[2] to role.
4. Uses an if statement with the and operator to check if location is "London" AND role is "Contractor".
5. If both conditions are met, use an **f-string** to print a message that includes the name, their role, and the location.
6. Otherwise, print "Screening Not Required".

Answer

```python
# initializes a variable for the list of records.
staff_audit = [
   ["London", "James", "Contractor"],
   ["Paris", "Marie", "Employee"],
```

```
    ["London", "Kiran", "Employee"],
    ["London", "Elena", "Contractor"]
    ]

# declares variables for items in the staff_audit records list.
# loops through the list to search for london and contractor
attributes.
# prints a message based on constraints.
for entry in staff_audit:
    location = entry[0]
    name = entry[1]
    role = entry[2]
    if location == "London" and role == "Contractor":
        print(f"Welcome to the team, {role} {name}! Enjoy the
{location} office.")
    else:
        print("Screening Not Required")
```

Data Aggregation and Frequency Mapping

> In this phase, I transitioned from simple list filtering to data aggregation. By utilizing Dictionaries, I learned how to track the frequency of specific events across a dataset. This is a critical skill for infrastructure monitoring, as it allows an engineer to transform a raw stream of hundreds of logs into a high-level summary of "Top Talkers" or "Primary Failure Points." This demonstrates the ability to provide executive-level insights from granular technical data.

Question 10: Building an Incident Frequency Map

This script demonstrates how to categorize and count events dynamically. Instead of manually searching for specific terms, the script builds a map of every unique event it finds and keeps a running tally.

Example

I have a list of support tickets by category. I want to count how many tickets exist for each category to see where the team is spending the most time.

Python

```python
# a list of ticket categories from the morning shift
tickets = ["Hardware", "Software", "Access", "Hardware",
"Hardware", "Access"]

# initializing an empty dictionary to store our counts
ticket_counts = {}

for category in tickets:
    if category in ticket_counts:
        # if the category is already in the dictionary, add 1 to
the count
        ticket_counts[category] = ticket_counts[category] + 1
    else:
        # if it's a new category, add it to the dictionary with a
count of 1
        ticket_counts[category] = 1

print(f"Daily Support Summary: {ticket_counts}")
```

Practice

You are reviewing security badge swipes for the Huntsville office. You need to create a summary that shows how many times each unique user accessed the "Secure Server Room."

Write a script that:

1. Starts with `access_logs = ["Sarah", "Joe", "Sarah", "Elena", "Sarah", "Joe", "James"]`.

2. Initializes an empty dictionary called `user_frequency`.
3. Loops through the `access_logs` list.
4. Uses an `if/else` statement to check if the `name` is already in the `user_frequency` dictionary.
5. If the name **is** in the dictionary, increment its value by 1.
6. If the name **is not** in the dictionary, add the name as a new key with a value of 1.
7. After the loop finishes, use an **f-string** to print: `"Access Audit Summary: {user_frequency}"`.

Answer

Python

```python
# declares a variable for list of names in access logs.
# declares an empty variable for user frequency.
access_logs = [
"Sarah",
"Joe",
"Sarah",
"Elena",
"Sarah",
"Joe",
"James"
]
user_frequency = {}

# loops through the names in the access logs.
# checks the frequency of each name in the list.
# prints an audit summary with the frequency of each name.
for name in access_logs:
    if name in user_frequency:
        user_frequency[name] = user_frequency[name] + 1
    else:
        user_frequency[name] = 1
print(f"Access Audit Summary: {user_frequency}")
```

System Integration: Diagnostic Returns and Logic Flow

> In this phase, I advanced from functions that simply print output to **Value-Returning Functions**. By utilizing the `return`keyword, I learned how to move data out of a function's local scope and back into the main program. This represents the "Integration" phase of the SDLC: a diagnostic module evaluates a condition and returns a signal (e.g., a Boolean `True/False`), which the parent system then uses to determine the next stage of execution. This is the foundation of building autonomous, self-healing infrastructure.

Question 12: Capturing Diagnostic Status

This script demonstrates the "Request/Response" pattern. The function acts as a security gatekeeper, evaluating an input and returning a status that determines system access.

Example

I want a tool that checks if a server temperature is within a safe range. Instead of just printing the temp, it returns a status so the system can decide whether to trigger emergency cooling.

Python

```python
def check_temp(temp):
    if temp < 80:
        return "NORMAL"
    else:
        return "CRITICAL"

# "catching" the return value in a variable
current_status = check_temp(85)

if current_status == "CRITICAL":
    print("Initiating emergency cooling sequence...")
```

Practice

You are checking for remote access. You need a function that checks a user's security clearance level and "returns" a boolean (`True` or `False`) to determine if they can run a diagnostic ping.

Write a script that:

1. Defines a function called `has_clearance` that takes one parameter: `level`.
2. Inside the function, check if `level` is greater than or equal to **5**.
3. If it is, **return** `True`. Otherwise, **return** `False`.
4. Outside the function, create a variable called `user_clearance` and set it to **7**.
5. Set a variable `access_granted` equal to the result of calling `has_clearance(user_clearance)`.
6. Use an `if` statement to print `"Diagnostic Tool Initialized"` if access is granted, or `"Access Denied"` if not.

Answer

Python

```python
# creates a function to give or deny access based on if a user's security level is greater than or equal to 5.
def has_clearance(level):
    if level >= 5:
        return True
    else:
        return False

# initializes a variable for a particular user's clearance level.
# catches the return value in a variable.
user_clearance = 7
access_granted = has_clearance(user_clearance)

# prints a message based on whether access is granted or denied.
if access_granted:
    print("Diagnostic Tool Initialized")
else:
```

```
    print("Access Denied")
```
